E DENSE SAE

To evaluate the effectiveness of SAE, we trained a comparison model, a *dense* sparse autoencoder (Dense SAE) without sparse constraints ($\lambda = 0$). The results show that the SAE consistently outperforms the Dense SAE across Accuracy, F1 score, Precision, and Recall, with a particularly significant improvement observed in Recall.

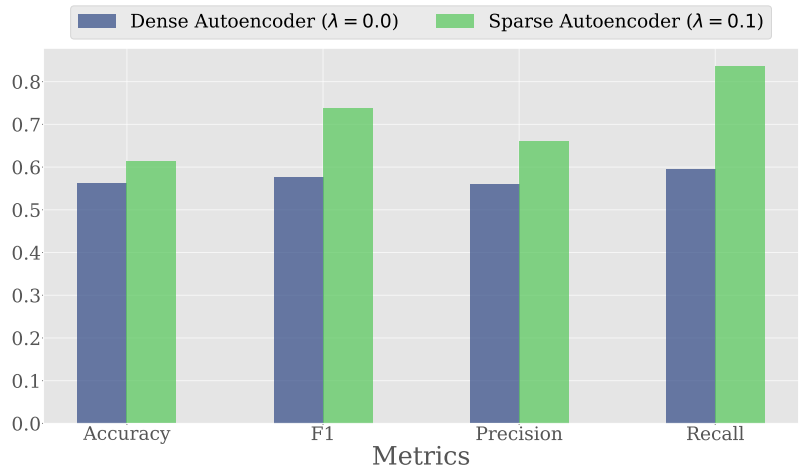

Figure 13: Comparison of evaluation metrics between dense autoencoder ($\lambda = 0$ and sparse autoencoder ($\lambda = 0.1$). Metrics include Accuracy, F1 score, Precision, and Recall. SAE consistently outperforms dense SAE across all metrics, with a particularly significant improvement in Recall.

F DATASET DETAILS

We show the some sample of PS-eval in Table 6. Each pair of sentences represents different meanings of the target word (e.g., "space" "save," "ball") or the same meaning. The label indicates whether the two contexts share the same meaning (Label: 1 for "Same") or differ (Label: 0 for "Different"). Poly-contexts demonstrate different meanings of the target word, while Mono-contexts show consistent meanings across sentences.

Table 6: Examples in PS-Eval.

Example Sentences	**Poly-contexts (space):** Context 1: *The astronauts walked in outer "space" without a tether.* Context 2: *The "space" between his teeth.* Label: 0 (**Different**)
	Mono-contexts (space): Context 1: *They stopped at an open "space" in the jungle.* Context 2: *The "space" between his teeth.* Label: 1 (**Same**)
	Poly-contexts (save): Context 1: *This will "save" you a lot of time.* Context 2: *She "save" the old family photographs in a drawer.* Label: 0 (**Different**)
	Poly-contexts (ball) Context 1: *The "ball" was already emptying out before the fire alarm sounded.* Context 2: *The "ball" rolled into the corner pocket.* Label: 0 (**Different**)

G F1 SCORE, PRECISION, RECALL

Figure 14 shows th performance evaluation of Sparse Autoencoders (SAEs) with varying expand ratios (8, 16, 32, 64, 128) across different metrics. The top row shows F1, Precision, and Recall plotted against Mean Squared Error (MSE), while the bottom row shows F1, Precision, and Recall plotted against L_0 sparsity. A lower MSE or L_0 indicates better performance, as seen with the trend of improved F1 scores and other metrics at specific expand ratios.

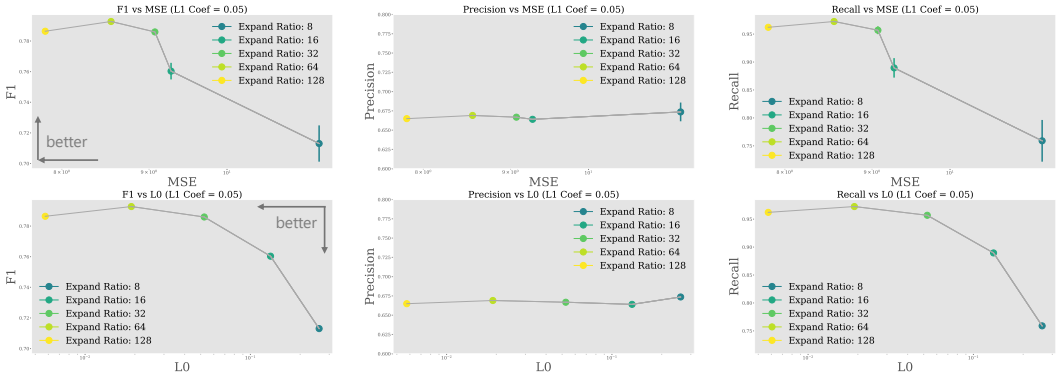

Figure 14: Comparison of F1 score, Precision, and Recall across different activation functions. The models were trained with JumpReLU (with varying thresholds), TopK (with different k), and standard ReLU. The results show that JumpReLU outperforms both TopK and standard ReLU in terms of F1 score, Precision, and Recall, indicating that it is better suited for capturing polysemantic representations.

H ACTIVATION FUNCTIONS

H.1 TOPK

The TopK activation function is designed to directly control sparsity in autoencoders by selecting only the k largest activations from the latent space, while zeroing out the rest. This approach eliminates the need for the commonly used L_1 penalty, which approximates sparsity by shrinking activations toward zero, often leading to suboptimal representations. The TopK function is defined as:

$$z = \text{TopK}(W_{\text{enc}}(x - b_{\text{pre}}))$$

where W_{enc} represents the encoder weights, x is the input vector, and b_{pre} is a bias term. By preserving only the top k largest values, TopK maintains a fixed level of sparsity, allowing for simpler model comparisons and more interpretable representations.

H.2 JUMPRELU

JumpReLU (Erichson et al., 2019) is an activation function introduced to enhance the performance of Sparse Autoencoders (SAEs) by improving the trade-off between sparsity and reconstruction fidelity. While the conventional ReLU activation function zeros out inputs less than or equal to zero, JumpReLU introduces a positive threshold θ. Specifically, JumpReLU zeros out pre-activations below this threshold, while retaining values above it. This mechanism helps in reducing the number of false positives, thus improving sparsity, while preserving the important feature activations with high fidelity. The function is defined as:

$$\text{JumpReLU}_\theta(z) = z \cdot H(z - \theta)$$

where H is the Heaviside step function and θ is a learnable threshold parameter. As illustrated in Figure 15, the JumpReLU function (blue) introduces a threshold $\theta = 1$, and only retains activations greater than this threshold, in contrast to ReLU (green), which activates any positive input.

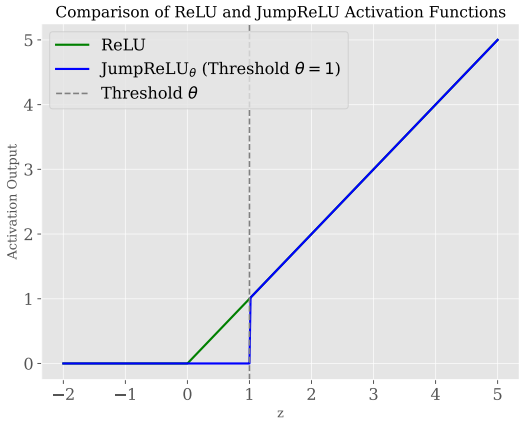

Figure 15: Comparison of ReLU and JumpReLU activation functions. The ReLU function (green) activates inputs greater than zero, while the JumpReLU function (blue) introduces a threshold $\theta = 1$, below which all inputs are zeroed out. The dashed gray line represents the threshold, illustrating how JumpReLU suppresses small activations, thereby promoting sparsity in activations.